DEVOPS FINAL CAPSTONE · SESSION 21

MediSlot: clinic scheduling, from laptop to a monitored EKS cluster



React · FastAPI · PostgreSQL · Docker · GitHub Actions · Trivy · Terraform · EKS · Helm · Argo CD · Prometheus · Grafana

Ayush Singh · github.com/ayushsingh199/DevOps_Capstone_Project

THE APPLICATION

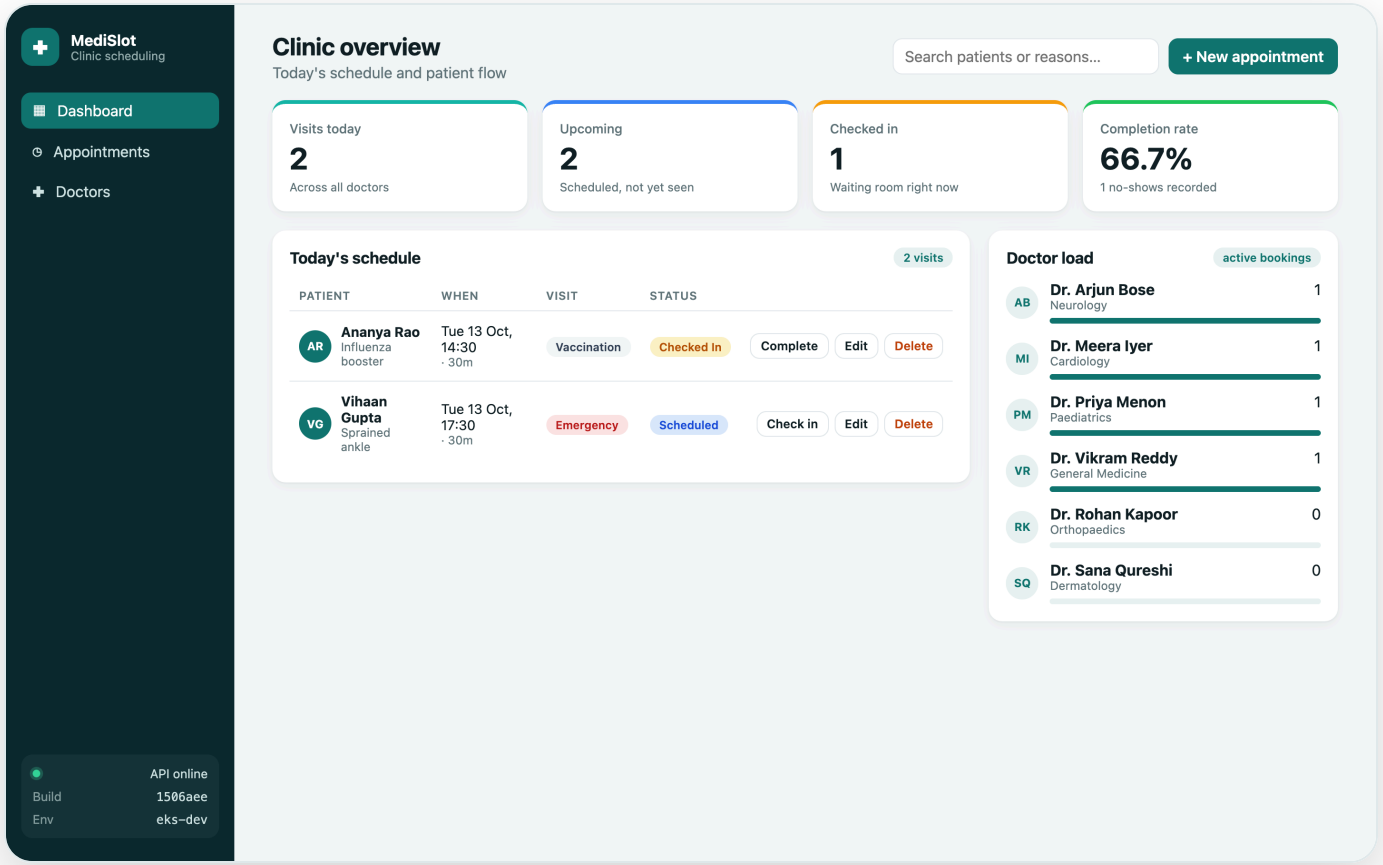
A front desk dashboard that refuses double-bookings

1. **Book**
   Patient, doctor, slot, visit type and reason, validated by Pydantic

2. **Check in → Complete**
   One click per status change; cancel and no-show are tracked

3. **No overlaps**
   An overlapping slot for the same doctor returns `409 Conflict`

4. **Live KPIs**
   Visits today, upcoming, waiting room, completion rate, doctor load



ARCHITECTURE

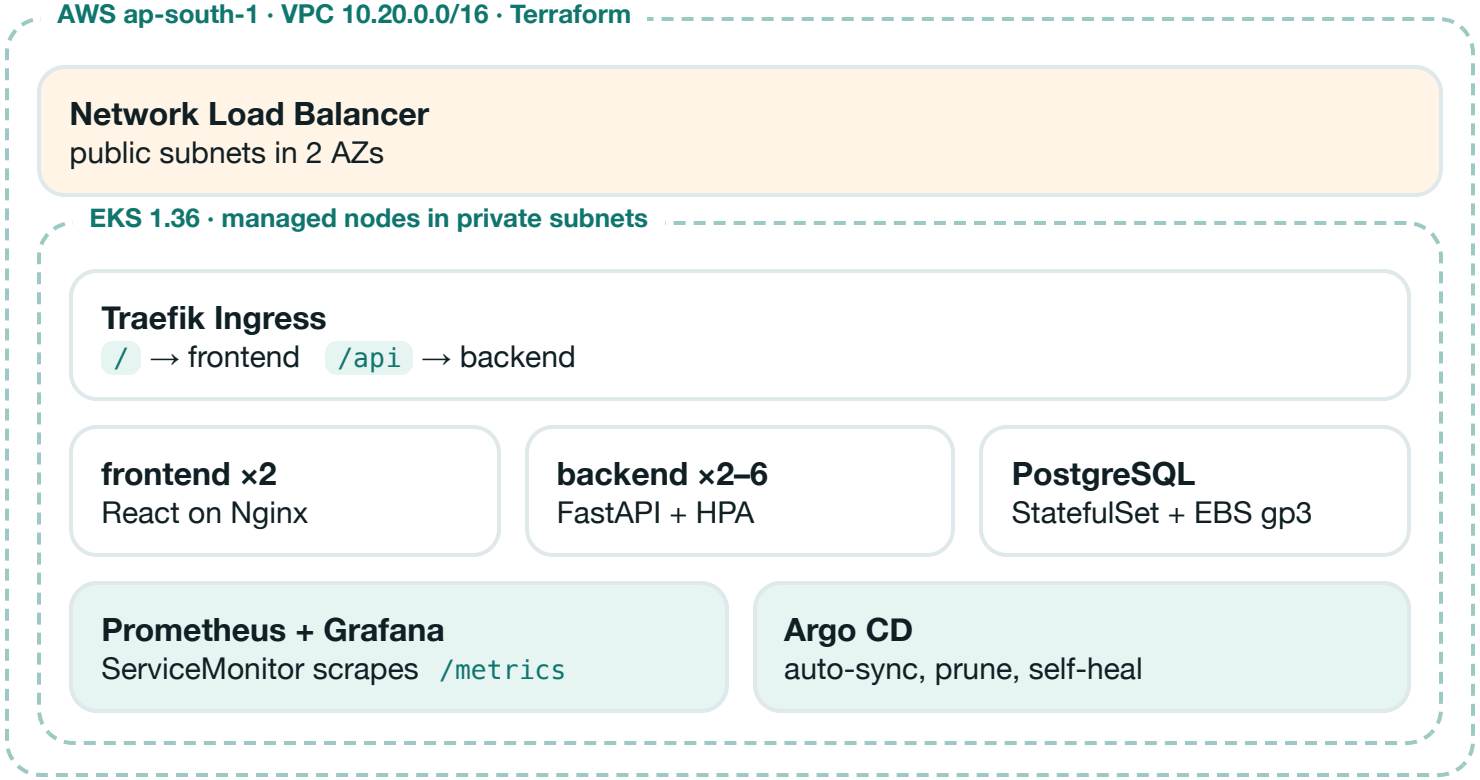
Two flows: requests in, releases pulled

1 · git push to main
Developer commit

2 · GitHub Actions
tests → build → Trivy gate → push `:git-sha` to GHCR

3 · GitOps commit
CI writes the new tag into `values-eks.yaml`

4 · Argo CD (in cluster)
pulls the change and does a rolling update



TECH STACK

One tool per job, all of it automated

LAYER	TECHNOLOGY	WHY
Application	React 19 + Vite · FastAPI · SQLAlchemy 2 · Alembic · PostgreSQL 16	Typed API, versioned schema
Quality	pytest (23 tests, 97% coverage) · ruff · ESLint · Prettier	First gate in the pipeline
Containers	Multi-stage Dockerfiles · non-root · Docker Compose	Small, reproducible images
CI/CD	GitHub Actions · GHCR · SHA-pinned actions	Every image traceable to a commit
Security	Trivy gate + SARIF · Pod Security "restricted" · NetworkPolicy	Defence in depth
Infrastructure	Terraform · AWS VPC · EKS · EBS CSI (Pod Identity)	Reviewable, repeatable infra
Delivery	Helm chart · Traefik Ingress · HPA · Argo CD	Pull-based GitOps
Observability	kube-prometheus-stack · ServiceMonitor · PrometheusRule · Grafana	See traffic, latency, errors

BACKEND AND DATABASE

Clean layers, migrations that can't race

REST API (FastAPI)

`GET / POST`	/api/appointments
`GET / PUT / DELETE`	/api/appointments/{id}
`GET`	/api/appointments/stats
`GET / POST / DELETE`	/api/doctors[/{id}]
`GET`	/health · /ready · /metrics · /api/info

Layering

`routers/` → HTTP · `services/` → rules · `models.py` → tables · `schemas.py` → contract

Alembic

`0001` creates **doctors** + **appointments** with check constraints and indexes; `0002` seeds the doctor roster. Runs in an initContainer under `pg_advisory_lock`.

Probes that mean something

`/health` = process alive (liveness) · `/ready` = database reachable (readiness, 503 otherwise)

TESTING AND CODE QUALITY

Bad code never reaches the registry

23

pytest cases across health, doctors, appointments, stats, metrics

97%

line coverage of the backend

0

tests that touch the production database

What the suite proves

- CRUD on every endpoint group, filters and search
- **409** on overlapping bookings, back-to-back slots allowed, cancelled slots rebookable
- **422** validation and **404** paths
- Alembic upgrade → downgrade round-trip

Isolation

`conftest.py` overrides `get_db` with a fresh in-memory SQLite per test. In CI, migrations also run against a **real PostgreSQL 16** service container.

Static checks

ruff (lint + format) · ESLint · Prettier. Any failure stops the pipeline.

DOCKER

Multi-stage, non-root, one command locally

backend/Dockerfile

`python:3.12-slim` builds wheels → slim runtime with OS packages upgraded and pip removed → runs as **UID 10001** with a HEALTHCHECK on `/health`

frontend/Dockerfile

`node:22-alpine` runs `npm ci && vite build` → `nginx-unprivileged` serves `dist/` as **UID 101** on port 8080. No Node in the final image.

docker-compose.yml

`postgres` (healthy) → `backend` (migrate + seed, healthy) → `frontend` on **localhost:3000**

```
$ docker compose up --build
✔ postgres Healthy
✔ backend  Healthy
✔ frontend Started → :3000

$ docker run --entrypoint id medislot-backend
uid=10001(app) gid=10001(app)
$ docker run --entrypoint id medislot-frontend
uid=101(nginx) gid=101(nginx)
```

CI/CD · GITHUB ACTIONS

Every push to main: test, build, scan, ship, deploy

gate 1

Backend

ruff · pytest + coverage · Alembic on real Postgres

→

gate 2

Frontend

npm ci · ESLint · vite build

→

Build

Buildx, GHA layer cache, matrix: backend + frontend

→

gate 3

Trivy

fail on fixable HIGH/CRITICAL · SARIF → Security tab

→

Push

`ghcr.io/…/medislot-*:<git-sha>`

→

Deploy

bump tags in `values-eks.yaml` → Argo CD

Traceable

Tags are the full commit SHA, never `latest`. The running SHA is shown in the app's sidebar.

Safe by default

Pull requests build and scan but never publish. Only scanned images reach GHCR.

Supply chain

All third-party actions are pinned to commit SHAs; a tag can be re-pointed, a SHA can't.

DEVSECOPS

Trivy gate: what it found and why it passes

What Trivy scanned

OS packages (Debian 13 backend, Alpine 3.23 frontend) and language dependencies (Python wheels) inside both images, against public vulnerability databases.

Result

0 fixable HIGH/CRITICAL in both images, so the gate passes and the images are pushed.

One CVE explained: CVE-2026-76642

HIGH, in Debian's `util-linux` (`bsdutils`, `libblkid1`), status *affected, no fix*. It's tolerated via `ignore-unfixed`: the API never runs these tools, and runs non-root, read-only, with no Linux capabilities. It's tracked in the Security tab and picked up by `apt-get upgrade` once Debian ships a fix.

Other layers

- Non-root images, multi-stage builds
- Namespace enforces Pod Security **restricted**
- `readOnlyRootFilesystem`, `drop: [ALL]`, seccomp
- NetworkPolicy: only the backend reaches Postgres
- Secrets generated in-cluster, never in Git
- Nodes in private subnets, EBS encrypted
- CI holds no cloud credentials (GitOps)

TERRAFORM · AWS

60 resources, one command, zero clicks

60

resources in `terraform plan`

4

subnets: 2 public (NLB, NAT) + 2 private (nodes)

1.36

EKS version · managed node group 2 × t3.large

vpc.tf

terraform-aws-modules/vpc: `10.20.0.0/16`, 2 AZs, single NAT gateway, subnet tags for internet-facing and internal load balancers

eks.tf + iam.tf

terraform-aws-modules/eks v21: public + private endpoint, creator admin via access entry; add-ons vpc-cni, coredns, kube-proxy, pod-identity, **EBS CSI** (Pod Identity role), **metrics-server**

Safe by design

Only `terraform.tfvars.example` is committed; state and tfvars are git-ignored. `scripts/teardown.sh` removes the NLB and EBS volumes first, then runs `terraform destroy`.

KUBERNETES · HELM

One chart, every environment

RESOURCE	DETAIL
Deployment backend	2–6 replicas · initContainers `wait-for-db` + `migrate`
Deployment frontend	2 replicas · read-only Nginx
StatefulSet postgres	2 Gi encrypted gp3 volume
Service ×3	ClusterIP: backend :8000, frontend :80, postgres :5432
Ingress	`/api` → backend · `/` → frontend
HPA · PDB · NetworkPolicy	CPU 60% · minAvailable 1 · DB isolation
ServiceMonitor · PrometheusRule	scrape config + 3 alerts

Values overlays

`values.yaml` defaults · `values-dev.yaml` local k3s · `values-eks.yaml` GHCR images, tags owned by CI

Ingress controller

An Ingress is only configuration. **Traefik** implements it (ingress-nginx was retired in March 2026); on EKS it sits behind an AWS NLB.

Zero-downtime rollouts

`maxUnavailable: 0` + readiness on `/ready`

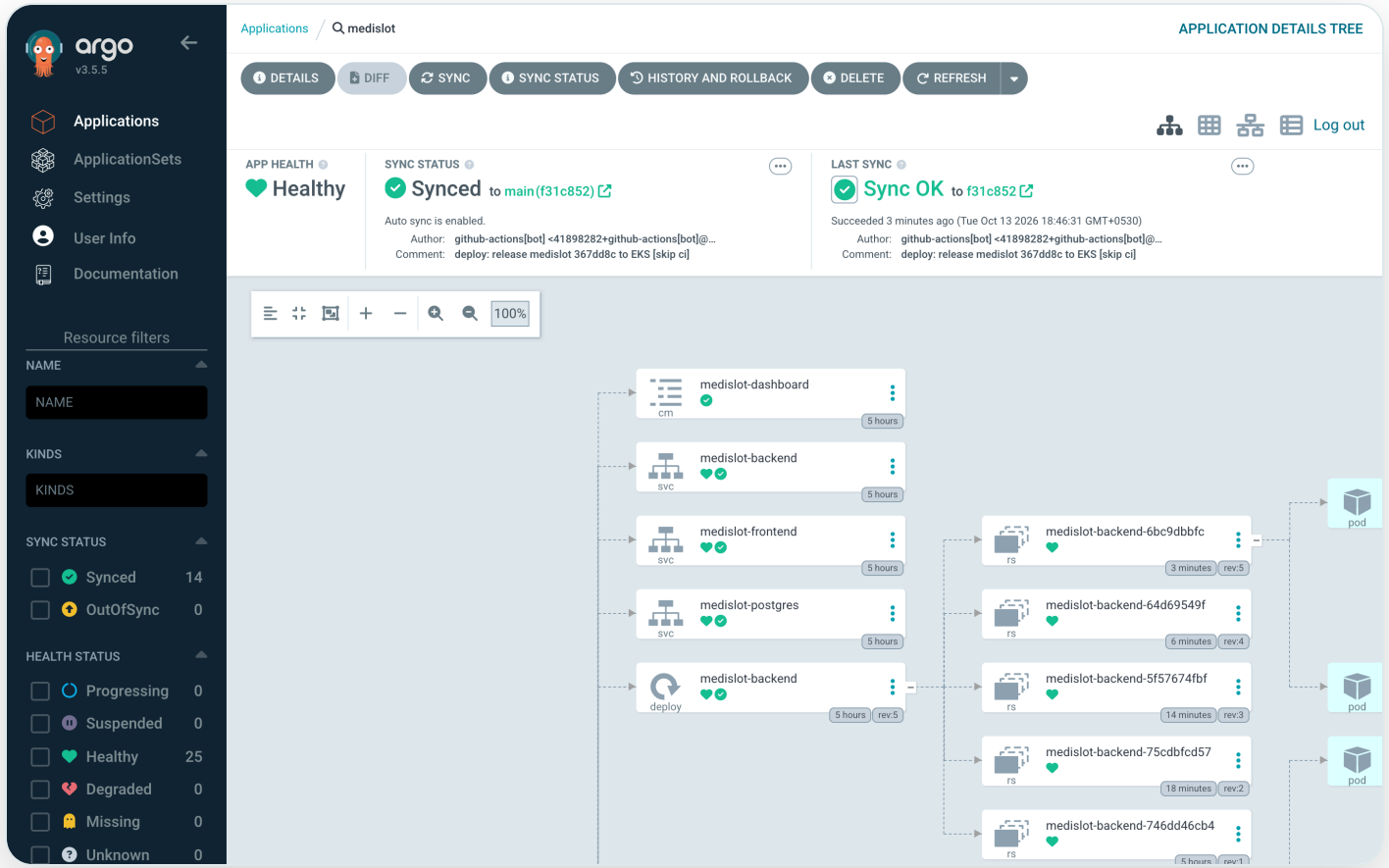
A release is a Git commit

1. **CI commits the new tag**
   "deploy: release medislot <sha>" in `values-eks.yaml`

2. **Argo CD detects the diff**
   polls `main` every 60 s, renders the Helm chart

3. **Rolling update**
   automated sync with prune and self-heal

↺ **Rollback = git revert**
manual `kubectl edit` drift is reverted automatically



OBSERVABILITY · PROMETHEUS + GRAFANA

Traffic, latency, errors and bookings, live

Metrics

`http_requests_total`, `http_request_duration_seconds` plus domain counters `medislot_appointments_booked_total`, `medislot_booking_conflicts_total`

Scraping

ServiceMonitor → every backend pod's `/metrics` every 15 s, all targets **UP**

Alerts

BackendDown · HighErrorRate > 5% · p95 > 500 ms



LESSONS LEARNED

What broke, and what I changed

An SCP beats AdministratorAccess

The first AWS account sat in an Organization whose service control policy denied EC2 and EKS. No IAM change can override that, so I moved to a standalone account.

New-account vCPU quota

5 vCPUs On-Demand, so 3 × t3.medium (6 vCPU) wouldn't launch. Resized to 2 × t3.large: 4 vCPU and more pods per node.

ingress-nginx retired

The community controller was retired in March 2026, so I chose Traefik while keeping a standard `networking.k8s.io/v1` Ingress.

Helm 4 wait semantics

The new "watcher" wait hung on a CRD-heavy chart; the bootstrap script now picks the right wait strategy for Helm 3 or 4.

Two pipelines, one tag file

Back-to-back pushes made two deploy jobs rebase the same lines and conflict. Releases are now serialized, superseded commits are skipped, and retries re-apply instead of rebasing.

Grafana OOMKilled

Under use, Grafana with ~25 bundled dashboards exceeded its 384 Mi limit (exit 137). Found it via `lastState.terminated` and raised the limit to 768 Mi.

LIVE DEMO

git push → tests → Trivy → GHCR → Argo CD → EKS → Grafana

Change
Edit the dashboard subtitle, commit, push

Watch
Pipeline goes green; Argo CD syncs the new SHA

Verify
New build SHA in the app, targets UP, live Grafana panels

Thank you · github.com/ayushsingh199/DevOps_Capstone_Project